

Comprendre et maintenir StockFlow

Ce document explique où se trouve chaque chose, comment le projet fonctionne et comment le modifier ou le réparer, même sans aide. Il accompagne le fichier `DOCUMENTATION.md` du dépôt.

Application de gestion de stock — React (Vite) + Tailwind · Node.js / Express + Prisma · PostgreSQL — 14 septembre 2026

Sommaire

1. Le projet en deux mots
2. Vue d'ensemble : comment les pièces s'emboîtent
3. Où se trouve quoi (arborescence commentée)
4. La base de données
5. Le backend (dossier `backend/`)
6. Le frontend (dossier `frontend/`)
7. Démarrer le projet au quotidien
8. Recettes : modifications courantes
9. Dépannage : problème → solution
10. Git : sauvegarder et mettre à jour le projet
11. Un jour, mettre en ligne pour de vrai (production)
12. Lexique des mots techniques

1. Le projet en deux mots

StockFlow est une application web de **gestion de stock multi-entrepôts** : produits, fournisseurs, clients, entrepôts, mouvements de stock (entrées, sorties, transferts), commandes d'achat et de vente, tableau de bord avec graphiques. On s'y connecte avec un email et un mot de passe.

Elle est composée de **deux applications** qui tournent en même temps :

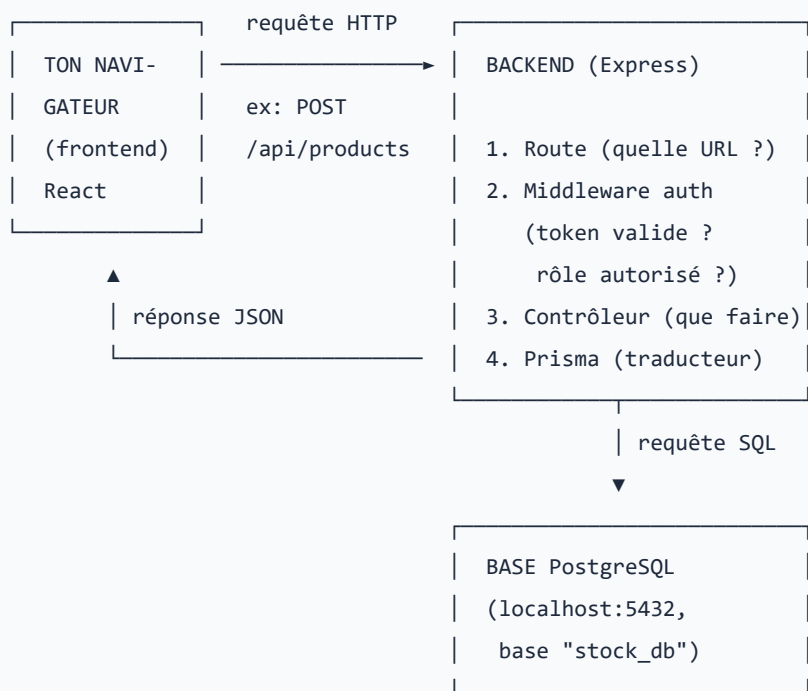
Application	Dossier	Technologie	Adresse locale
Le backend (l'API)	backend/	Node.js + Express + Prisma	http://localhost:4000
Le frontend (l'écran)	frontend/	React (Vite) + Tailwind CSS	http://localhost:5173

La **base de données** est **PostgreSQL**, installée sur ton ordinateur. Elle stocke tout : produits, quantités, utilisateurs, commandes...

💡 **Règle d'or** : le frontend **ne parle jamais directement à la base de données**. Il envoie des requêtes HTTP au backend, et seul le backend lit ou écrit dans la base. C'est ce qui garantit la sécurité et la cohérence des stocks.

2. Vue d'ensemble : comment les pièces s'emboîtent

Quand tu cliques sur « Enregistrer » dans une page du site, voici le voyage :



- La réponse est du **JSON** (du texte structuré que React sait afficher).
- L'authentification utilise un **JWT** : à la connexion, le backend délivre un « badge » (token) que le frontend présente à chaque requête suivante.

3. Où se trouve quoi (arborescence commentée)

```
stock-app/
|
├─ .gitignore          ← liste des fichiers que git n'envoie JAMAIS sur GitHub
|                        (.env, node_modules, dist...)
├─ README.md           ← présentation courte du projet
├─ DOCUMENTATION.md    ← la même aide que ce PDF, en version texte
|
├─ backend/            ← ===== L'API (serveur) =====
|   │ .env              ← ⚠ SECRETS (mot de passe BDD, clé JWT). JAMAIS sur GitHub.
|   │ .env.example      ← copie sans secrets, modèle à suivre
|   │ package.json      ← liste des dépendances + scripts npm du backend
|   │ package-lock.json ← versions exactes installées (à versionner)
|   │ prisma/
|   │ │ schema.prisma   ← ★ LA STRUCTURE DE LA BASE (toutes les tables)
|   │ │ migrations/    ← historique des modifications de la base (versionné)
|   │ │ │ 20260914002548_init/
|   │ │ │ seed.js       ← crée les comptes + données de démonstration
|   └─ src/
|       │ index.js      ← ★ point d'entrée : démarre Express, branche les routes
|       │ lib/prisma.js ← connexion unique à la base (partagée partout)
|       │ middleware/
|       │ │ auth.js      ← vérifie le token JWT et les rôles (ADMIN/MANAGER/...)
|       │ │ │ errorHandler.js ← attrape les erreurs → réponse JSON propre
|       │ │ routes/      ← 1 fichier par domaine : "si l'URL est /api/xxx,
|       │ │ │             appelle tel contrôleur"
|       └─ controllers/  ← ★ LA LOGIQUE MÉTIER (le "que faire" de chaque route)
|           │ auth.controller.js      (login, profil)
|           │ genericCrud.js          (fabrique de CRUD réutilisable)
|           │ category.controller.js  (3 lignes : utilise genericCrud)
|           │ supplier / customer / warehouse.controller.js (idem)
|           │ product.controller.js   (logique spécifique produits)
|           │ stock.controller.js      (mouvements, transactions)
|           │ purchaseOrder.controller.js (réception achat → stock +)
|           │ salesOrder.controller.js (vente → stock -, annulation → stock +)
|           └─ dashboard.controller.js (statistiques + graphiques)
```

- └─ frontend/
 - └─ .env ← ===== L'APPLICATION VISIBLE =====
 - └─ .env.example ← adresse de l'API (VITE_API_URL)
 - └─ package.json ← modèle
 - └─ vite.config.js ← dépendances + scripts npm du frontend
 - └─ vite.config.js ← configuration de l'outil de build Vite
 - └─ tailwind.config.js ← couleurs et styles du design system
 - └─ postcss.config.js ← pipeline CSS (Tailwind)
 - └─ index.html ← page HTML unique (React s'y installe)
 - └─ dist/ ← résultat du "npm run build" (généré, ignoré par git)
 - └─ src/
 - └─ main.jsx ← point d'entrée React
 - └─ App.jsx ← ★ LES ROUTES (quelle page pour quelle URL)
 - └─ index.css ← styles globaux + classes utilitaires Tailwind
 - └─ api/
 - └─ client.js ★ axios configuré : ajoute le token, gère la déconnexion
 - └─ context/
 - └─ AuthContext.jsx ← qui est connecté ? login/logout (partagé partout)
 - └─ utils/
 - └─ format.js ← formatage argent/dates, libellés des statuts et rôles
 - └─ components/ ← briques réutilisables de l'interface
 - └─ Layout.jsx (menu latéral + barre du haut)
 - └─ DataTable.jsx (tableau avec tri)
 - └─ Modal.jsx, ConfirmDialog.jsx, Toast.jsx, Badge.jsx, Button.jsx,
 - └─ Field.jsx, PageHeader.jsx, StatCard.jsx, icons.jsx
 - └─ pages/ ★ 1 fichier = 1 écran
 - └─ Login.jsx, Dashboard.jsx, Products.jsx, Categories.jsx,
 - └─ Suppliers.jsx, Customers.jsx, Warehouses.jsx,
 - └─ StockMovements.jsx, PurchaseOrders.jsx, SalesOrders.jsx

4. La base de données

Où sont les données ?

Dans **PostgreSQL**, installé localement. Les identifiants sont dans `backend/.env` (ligne `DATABASE_URL`) :

```
postgresql://postgres:postgres@localhost:5432/stock_db?schema=public
└─user─┐ └─pass─┐ └─base─┐
```

Prisma : le traducteur entre JavaScript et SQL

On n'écrit pas de SQL à la main. **Prisma** lit le fichier `backend/prisma/schema.prisma` (la description des tables) et génère un client JavaScript. Tu as un modèle (une « table ») par concept :

Modèle (schema.prisma)	Table SQL	Rôle
User	users	comptes, rôle ADMIN / MANAGER / EMPLOYEE
Warehouse	warehouses	entrepôts et boutiques
Category / Supplier / Customer	categories / suppliers / customers	référentiels
Product	products	catalogue (SKU unique, prix achat/vente, seuil d'alerte)
StockItem	stock_items	quantité d'un produit dans un entrepôt
StockMovement	stock_movements	historique IN / OUT / TRANSFER / ADJUSTMENT
PurchaseOrder + PurchaseOrderItem	purchase_orders ...	commandes fournisseurs
SalesOrder + SalesOrderItem	sales_orders ...	commandes clients

💡 Les quantités ne sont jamais devinées : `StockItem` fait foi, et chaque changement laisse une trace dans `StockMovement`.

Les commandes Prisma (à lancer dans `backend/`)

Commande	À quoi ça sert
<code>npx prisma migrate dev</code>	applique les changements du <code>schema.prisma</code> à la base (crée une « migration »)
<code>npm run seed</code>	crée/re-crée les données de démonstration (idempotent : pas de doublons)
<code>npx prisma studio</code>	ouvre un explorateur visuel de la base sur <code>http://localhost:5555</code> — très pratique pour vérifier des données
<code>npx prisma generate</code>	régénère le client après un changement de schéma (fait automatiquement par <code>migrate dev</code>)

⚠ **Ne supprime jamais le dossier `prisma/migrations/`** : c'est la mémoire des modifications de la base. Il doit être versionné sur GitHub.

Le trajet d'une requête

Exemple : `GET /api/products` (liste des produits).

1. `src/index.js` — Express reçoit la requête, voit le préfixe `/api/products` et délègue à `routes/product.routes.js`.
2. `src/middleware/auth.js` — `authenticate` vérifie le header `Authorization: Bearer <token>` et décode le token JWT → `req.user` (`{ id, role, email }`). Certaines routes ajoutent `authorize("ADMIN", "MANAGER")`.
3. `src/routes/product.routes.js` — pour `GET /` appelle `ctrl.list`.
4. `src/controllers/product.controller.js` — exécute les requêtes Prisma, calcule le stock total par produit, renvoie le JSON.
5. `src/middleware/errorHandler.js` — si une étape a planté, renvoie une erreur JSON lisible au lieu de faire crasher le serveur.

Le CRUD générique : le truc malin du projet

`controllers/genericCrud.js` fabrique les 5 opérations standards (liste / détail / création / modification / suppression) pour **n'importe quel modèle simple**. C'est pourquoi `category.controller.js` ne fait que 6 lignes :

```
const createCrudController = require("../genericCrud");
module.exports = createCrudController("category", { searchFields: ["name"] });
```

Les contrôleurs **spécifiques** (product, stock, purchaseOrder, salesOrder, dashboard, auth) contiennent de la vraie logique métier.

La logique du stock (le cœur du métier)

Tout ce qui touche aux quantités est fait dans une **transaction Prisma** (`prisma.$transaction`) : soit toutes les écritures réussissent, soit aucune — impossible d'avoir un stock à moitié mis à jour.

- **Vente créée** → vérifie le stock disponible dans l'entrepôt, sinon erreur « stock insuffisant » → décrémente `StockItem` → crée un mouvement `OUT`.
- **Vente CONFIRMED annulée** → recrée la quantité → mouvement de contrepassation.
- **Achat réceptionné** → incrémente `StockItem` de l'entrepôt → mouvement `IN`.
- **Mouvement manuel** → `IN` (entrée), `OUT` (sortie), `TRANSFER` (d'un entrepôt vers un autre), `ADJUSTMENT` (quantité finale signée `+` / `-`).

Les variables d'environnement (`backend/.env`)

Variable	Rôle	Peut changer ?
<code>DATABASE_URL</code>	connexion PostgreSQL	seulement si tu changes le mot de passe PostgreSQL
<code>PORT</code>	port de l'API (4000)	si 4000 est déjà pris
<code>JWT_SECRET</code>	clé qui signe les tokens	⚠ à changer pour la production, jamais publié
<code>JWT_EXPIRES_IN</code>	durée de validité du token (7d)	oui
<code>CLIENT_URL</code>	adresse du frontend autorisée (CORS)	si le frontend change d'adresse

6. Le frontend (dossier `frontend/`)

L'ossature

- `main.jsx` → monte React dans `index.html`, active le routeur et `AuthProvider`.
- `App.jsx` → **la carte des URLs**. `/login` est publique ; toutes les autres pages sont enveloppées dans `ProtectedRoute` (renvoie vers `/login` si non connectée) et affichées dans `Layout` (menu latéral).
- `context/AuthContext.jsx` → stocke l'utilisateur connecté. À la connexion, le token et l'utilisateur sont gardés dans le **localStorage** du navigateur (clés `stockflow_token` et `stockflow_user`), donc la session survit au rechargement de la page.
- `api/client.js` → axios pré-configuré : ajoute automatiquement le token à chaque requête ; si l'API répond 401 (session expirée), nettoie le stockage et renvoie à `/login`. L'adresse de l'API vient de `VITE_API_URL` (`frontend/.env`), avec `http://localhost:4000/api` par défaut.

Structure d'une page type (ex : `pages/Products.jsx`)

Toutes les pages de gestion suivent le même schéma — si tu sais lire une, tu sais les lire toutes :

- des `useState` pour les données, le chargement, la modale ouverte, le formulaire ;
- un `useEffect` qui charge la liste via `client.get("/products")` au montage ;
- un formulaire dans une `Modal` qui fait `client.post` / `client.put` ;
- un bouton supprimer avec `ConfirmDialog` puis `client.delete` ;
- un rendu avec `PageHeader` + `DataTable` + `Toast` pour les confirmations.

Les composants réutilisables (`components/`)

Ne réinvente rien : `Button`, `Field` (champ de formulaire), `Modal`, `ConfirmDialog`, `DataTable`, `Badge` (étiquettes colorées), `StatCard` (carte de statistique), `Toast` (notification), `PageHeader`, `icons.jsx` (icônes SVG), `Layout` (squelette de l'app avec le menu).

Le formatage (`utils/format.js`)

Argent, dates, libellés des statuts (« PENDING » → « En attente ») et rôles.

💡 **La monnaie affichée** : c'est la constante `CURRENCY_SUFFIX = "F"` en haut de `frontend/src/utils/format.js`. Change-la en `" FCFA"`, `" €"` ... pour adapter l'affichage partout.

7. Démarrer le projet au quotidien

Conditions : PostgreSQL doit être démarré (le service Windows « postgresql »).

Terminal 1 — l'API :

```
cd C:\Users\Hortencia\Documents\GitHub\stock-app\backend
npm run dev
# → "API démarrée sur http://localhost:4000"
```

Terminal 2 — l'interface :

```
cd C:\Users\Hortencia\Documents\GitHub\stock-app\frontend
npm run dev
# → ouvre http://localhost:5173 dans le navigateur
```

Comptes de démonstration (créés par `npm run seed`) :

Rôle	Email	Mot de passe
ADMIN	<code>admin@stock.app</code>	<code>admin123</code>
MANAGER	<code>manager@stock.app</code>	<code>staff123</code>
EMPLOYEE	<code>employee@stock.app</code>	<code>staff123</code>

Vérifications rapides : `http://localhost:4000/api/health` doit répondre `{"status":"ok"}`. Pour arrêter un serveur : `Ctrl + C` dans son terminal.

8. Recettes : modifications courantes

Recette A — Ajouter un champ (ex : « garantie en mois » sur les produits)

1. Ouvre `backend/prisma/schema.prisma`, dans le modèle `Product` ajoute :

```
warrantyMonths Int @default(0)
```

2. Dans `backend/` : `npx prisma migrate dev --name ajout_garantie` (la colonne est créée dans la base, le client Prisma est régénéré).

3. Frontend : dans `frontend/src/pages/Products.jsx`, ajoute le champ dans le formulaire (`Field`) et, si tu veux l'afficher, une colonne dans le tableau. C'est tout : l'API enregistrera le champ car le contrôleur transmet `req.body` .

🔑 Le principe est **toujours le même** : schéma → `migrate dev` → ajuster la page frontend.

Recette B — Ajouter une entité complète (ex : « Sites de production »)

Backend (copie le modèle des catégories, le plus simple) :

1. `schema.prisma` : ajoute le modèle `ProductionSite` (inspire-toi de `Category`).

2. `npx prisma migrate dev --name production_sites`

3. Crée `src/controllers/productionSite.controller.js` :

```
const createCrudController = require("../genericCrud");
module.exports = createCrudController("productionSite", { searchFields: ["name"] });
```

4. Crée `src/routes/productionSite.routes.js` (copie `category.routes.js`, remplace le nom) et branche-le dans `src/index.js` :

```
const productionSiteRoutes = require("../routes/productionSite.routes");
app.use("/api/production-sites", productionSiteRoutes);
```

Frontend (copie `pages/Categories.jsx`, le plus simple) :

5. Crée `src/pages/ProductionSites.jsx` en l'adaptant (l'URL d'API : `/production-sites`).

6. Déclare la route dans `src/App.jsx` et le lien dans le menu dans `components/Layout.jsx` .

Recette C — Créer un utilisateur / changer un mot de passe

Le plus simple : `npx prisma studio` (dans `backend/`) → table `users` . Le mot de passe doit être **haché** : pour le recalculer, colle ceci dans un terminal (toujours dans `backend/`) et remplace la colonne `password` par le résultat :

```
node -e "const b=require('bcryptjs');console.log(b.hashSync('NouveauMotDePasse',10))"
```

Recette D — Repartir d'une base propre

```
cd backend
npx prisma migrate reset # efface TOUT et rejoue migrations + seed
```

⚠ Efface les vraies données — à ne faire qu'en développement.

Recette E — Vérifier ou corriger des quantités

Ouvre `npx prisma studio`, regarde `stock_items` (les quantités) et `stock_movements` (l'historique). Pour corriger une quantité à la main, crée plutôt un mouvement `ADJUSTMENT` depuis l'application : la trace est conservée.

9. Dépannage : problème → solution

Symptôme	Cause probable	Solution
« API démarrée » ne s'affiche pas / erreur <code>EADDRINUSE</code>	le port 4000 est déjà pris (une ancienne instance tourne)	ferme l'ancien terminal, ou change <code>PORT</code> dans <code>backend/.env</code>
Erreur de connexion Prisma / code <code>P1001</code>	PostgreSQL n'est pas démarré, ou mauvais mot de passe dans <code>.env</code>	démarre le service PostgreSQL (Windows : Services → <code>postgresql-x64-...</code> → Démarrer)
Le site affiche « Erreur serveur » ou rien ne charge	le backend n'est pas lancé	lance le terminal 1 (§7)
<code>{"status": "ok"}</code> ne répond pas	API arrêtée ou mauvais port	vérifie le terminal 1 et <code>PORT</code>
Login : « Identifiants invalides »	le seed n'a jamais tourné (ou email/mot de passe faux)	<code>npm run seed</code> dans <code>backend/</code> , puis utilise les comptes du §7
Une page redirige sans arrêt vers <code>/login</code>	token expiré (7 jours) ou invalide	déconnecte/reconnecte-toi ; vide le localStorage (F12 → Application) si besoin
Erreur CORS dans la console du navigateur	le frontend tourne sur une autre adresse que <code>CLIENT_URL</code>	aligne <code>CLIENT_URL</code> (<code>backend/.env</code>) avec l'adresse réelle du frontend
« Accès refusé pour ce rôle » (403)	le compte n'a pas le rôle requis (ex : suppression réservée à l'ADMIN)	connecte-toi avec le compte ADMIN
Page blanche après une modification du frontend	erreur JavaScript	regarde la console du navigateur (F12) et le terminal Vite
Quantités incohérentes	manipulation directe en base	refais un <code>ADJUSTMENT</code> depuis l'app ; en dernier recours <code>npm run seed</code> recalcule la démo
<code>prisma migrate dev</code> propose un reset	la base a dérivé des migrations	accepte le reset seulement en développement (données de démo perdues)

10. Git : sauvegarder et mettre à jour le projet

Le projet vit sur GitHub. Trois gestes à connaître :

Sauvegarder ton travail (après chaque séance de travail) :

```
git add .                # prépare tous les changements
git commit -m "description du changement"  # crée un point de sauvegarde local
git push                 # envoie sur GitHub
```

Récupérer les dernières modifications (sur un autre ordinateur, ou après un changement fait par quelqu'un d'autre) :

```
git pull
```

Voir l'état courant :

```
git status              # quels fichiers ont changé ?
git log --oneline       # l'historique des sauvegardes
```

⚠ Les fichiers listés dans `.gitignore` (`node_modules/` , `.env` , `dist/`) ne seront jamais envoyés sur GitHub — c'est voulu. Une personne qui récupère le projet doit recréer `.env` à partir de `.env.example` puis lancer `npm install` dans `backend/` et `frontend/`, puis `npx prisma migrate dev` et `npm run seed` dans `backend/`.

11. Un jour, mettre en ligne pour de vrai (production)

Pour une vraie utilisation (au-delà de ton PC), il faudra :

1. **Changer** `JWT_SECRET` dans le `.env` du serveur (une longue chaîne aléatoire).
2. **Changer les mots de passe des comptes de démo** (voir Recette C) ou supprimer les comptes de démo.
3. **Créer un utilisateur ADMIN avec un mot de passe fort** pour toi.
4. **Servir le frontend compilé** : `npm run build` dans `frontend/` produit le dossier `dist/` à héberger (Netlify, Vercel, un serveur nginx...), avec `VITE_API_URL` pointant vers l'adresse publique de l'API.
5. **Héberger l'API + PostgreSQL** (Railway, Render, un VPS...) avec les bonnes valeurs dans le `.env` de production (`DATABASE_URL` , `CLIENT_URL`).
6. **HTTPS** activé (la plupart de ces plateformes le font automatiquement).

12. Lexique des mots techniques

Mot	Signification simple
API	le « serveur » : un programme qui répond à des questions (requêtes HTTP)
Endpoint / route	une « question » précise, ex : <code>GET /api/products</code> = « donne-moi les produits »
JSON	format texte structuré utilisé pour les échanges entre frontend et backend
JWT / token	badge numérique remis à la connexion, présenté à chaque requête
CRUD	Create, Read, Update, Delete — les 4 opérations de base sur des données
Middleware	étape intermédiaire par laquelle passe une requête (ex : vérification du token)
Migration	fichier qui décrit une modification de la structure de la base
Seed	script qui remplit la base avec des données de départ / de démonstration
ORM (Prisma)	outil qui permet de parler à la base en JavaScript au lieu de SQL
Transaction	groupe d'opérations « tout ou rien » pour garder des données cohérentes
CORS	règle de sécurité : quelles adresses web ont le droit d'appeler l'API
localStorage	petite mémoire du navigateur (on y garde le token de session)
Composant	brique d'interface réutilisable (bouton, modale, tableau...)
Build	compilation du frontend en fichiers optimisés (dossier <code>dist/</code>)

StockFlow — Guide de maintenance · Document généré le 14/09/2026. Pense à le mettre à jour quand le projet évolue !